

Motor de Căutare Simplu pentru Documente Text

-Proiect POO-

Nume, prenume: Blîndu Constantin

Grupa: 3123B

CUPRINS

Motor de Căutare Simplu pentru Documente Text	1
1. Introducere temă și obiective proiect	3
2. Proiectul din punct de vedere al utilizatorului	4
Alegerea directorului, compilarea și rularea	4
Prezentare interfață utilizator	4
3. Proiectul din punct de vedere al programatorului	6
Clasele utilizate	6
Concepte POO utilizate	9
4. Diagrama UML	11

1. Introducere temă și obiective proiect

În acest proiect, tema realizată a fost un motor de căutare simplu pentru documente text. Cerințele obligatorii ale acestuia au fost:

- Clasa Document (caleFisier, conținut).
- Clasa Index care stochează un map> reprezentând cuvânt -> documente în care apare.

Practic, trebuia să ne folosim de cunoștințele de POO dobândite pentru a finaliza tema propusă.

Obiectivele principale ale proiectului realizate sunt:

- încărcarea documentelor dintr-un director, se efectueaza la momentul build-ului,
- construirea indexului, caută un cuvânt și returnează documentele relevante (va urma un exemplu afisat in sectiunea „Program d.p.d.v. al utilizatorului)
- citirea din fișiere și manipularea stringurilor.

Obiectivele secundare realizate sunt:

- Utilizarea design pattern-ului Observer:
- Document este observabil, Logger este observer și înregistrează fiecare căutare.
- Opțiuni de căutare avansată (AND, OR între cuvinte).
- Eliminarea de stop-words ("și", "în", etc.) înainte de indexare.
- Teste unitare pentru logica și validării tranzacțiilor.

2. Proiectul din punct de vedere al utilizatorului

Alegerea directorului, compilarea și rularea

În primul rând, utilizatorul este nevoit să parcurgă acești pași pentru a rula proiectul în Linux.

- **Alegerea directorului** se face cu comanda `cd` și introducerea numelui ce aparține folderului.

În acest caz, este:

`cd ~/motor_cautare`

- **Compilarea** efectuează acțiunea de „build” necesară pentru a rula programul. Se poate realiza prin două metode:

- Manual

`g++ -std=c++17 -Wall -O2 -o motor_cautare motor_cautare.cpp`

`g++ -std=c++17 -Wall -O2 -o teste teste_unitare.cpp`

- Makefile

`make all`

`make test (compilare + rulare teste)`

- **Rularea** se realizează astfel:

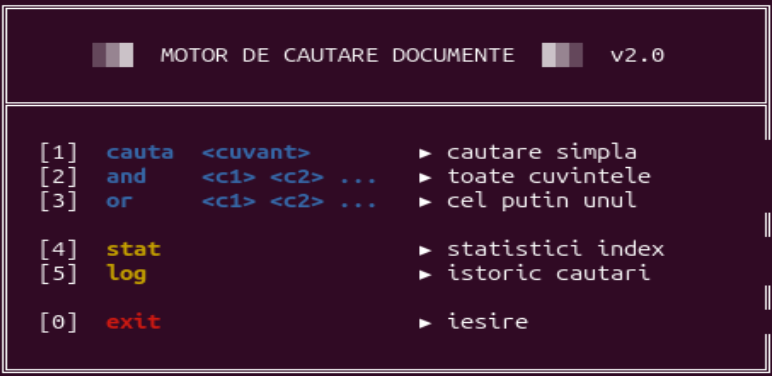
`./teste`

`./motor_cautare ./documente`

Prezentare interfață utilizator

În imaginea următoare este prezentată interfața cu care utilizatorul va lucra:

```
constantin@constantin-VMware-Virtual-Platform:~/motor_cautare$ ./motor_cautare documente
[INFO] Inregistrari log -> search.log
[INFO] 8 document(e) incarcate.
[INFO] Index construit cu 148 cuvinte unice (stop-words exclude).
```



```
[1] cauta <cuvant>      ▶ cautare simpla
[2] and  <c1> <c2> ...   ▶ toate cuvintele
[3] or   <c1> <c2> ...   ▶ cel puțin unul

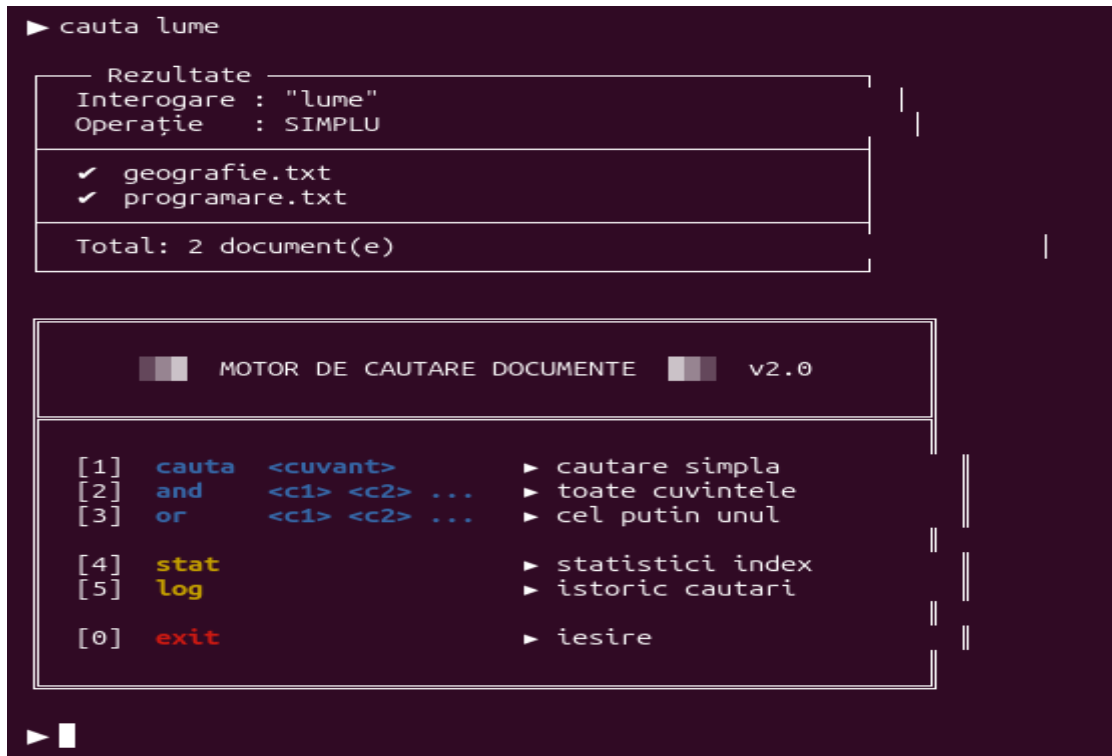
[4] stat                ▶ statistici index
[5] log                 ▶ istoric cautari

[0] exit                ▶ iesire
```

Înainte de a utiliza motorul de căutare, se pot observa 3 acțiuni ce au avut loc:

- Comenzile acceptate sunt reținute într-un fișier de tip log.
- Se menționează numărul de documente încărcate.
- Se construiește indexul cu toate cuvintele unice distincte de cele stop-words.

La alegerea unei comenzi, consola va arăta asemănător cu următoarea imagine:



```
► cauta lume

Rezultate
Interogare : "lume"
Operație   : SIMPLU

✓ geografie.txt
✓ programare.txt

Total: 2 document(e)

MOTOR DE CAUTARE DOCUMENTE v2.0

[1] cauta <cuvant>          ► cautare simpla
[2] and   <c1> <c2> ...     ► toate cuvintele
[3] or    <c1> <c2> ...     ► cel puțin unul

[4] stat                                     ► statistici index
[5] log                                      ► istoric cautari

[0] exit                                   ► iesire

►
```

3.Proiectul din punct de vedere al programatorului

Clasele utilizate

- EventCautare — obiect de date (struct), este utilizat pentru a transmite informatia de la Index la Logger (lipsit de metode):

```
struct EventCautare {  
    std::string interogare;          // textul cautat  
    std::string tipOperatie;         // "AND" | "OR" | "SIMPLU"  
    std::vector<std::string> rezultate; // fisierele gasite  
    std::string timestamp;  
};
```

- StopWords – clasă ce acționează ca un filtru de cuvinte.

```
class StopWords {  
public:  
    std::unordered_set<std::string> lista;  
  
    StopWords() {  
        // Romana  
        for (const char* sw : {  
            "si", "in", "la", "pe", "cu", "de", "un", "o", "a", "al", "ale",  
            "cel", "cea", "cei", "cele", "din", "spre", "prin", "pentru",  
            "care", "ce", "sa", "se", "nu", "dar", "sau", "ori", "ca", "fi",  
            "este", "sunt", "era", "au", "am", "ai", "ar", "fi", "fost",  
            "acest", "aceasta", "acesta", "acestia", "acestea",  
            "acel", "aceea", "acei", "acele", "eu", "tu", "el", "ea",  
            "noi", "voi", "ei", "ele", "ma", "te", "il", "le", "ne", "va",  
            "le", "imi", "iti", "ii", "ne", "vi", "le", "li", "isi",  
            "mai", "tot", "toate", "toti", "toata", "foarte", "acum",  
            "atunci", "dupa", "inainte", "intre", "asupra", "despre",  
            "daca", "cat", "cand", "unde", "cum", "astfel", "deci",  
            "iar", "insa", "totusi", "chiar", "doar", "poate", "fie"  
        }) lista.insert(sw);  
  
        // Engleza  
        for (const char* sw : {  
            "the", "a", "an", "and", "or", "but", "in", "on", "at", "to",  
            "for", "of", "with", "by", "from", "is", "are", "was", "were",  
            "be", "been", "being", "have", "has", "had", "do", "does", "did",  
            "will", "would", "shall", "should", "may", "might", "must",  
            "can", "could", "not", "no", "nor", "so", "yet", "both",  
            "either", "neither", "each", "every", "all", "any", "few",  
            "more", "most", "other", "some", "such", "than", "that", "this",  
            "these", "those", "i", "you", "he", "she", "it", "we", "they",  
            "what", "which", "who", "whom", "whose", "when", "where", "why", "how"  
        }) lista.insert(sw);  
    }  
};
```

- IObserver — interfață abstractă : Stabilește ce metodă trebuie să ofere orice observer

```
/**
 * Interfata Observer - orice clasa care vrea sa fie notificata
 * la o cautare trebuie sa implementeze onCautare().
 */
class IObserver {
public:
    virtual ~IObserver() = default;
    virtual void onCautare(const EventCautare& ev) = 0;
};

/**
 * Interfata Observable - Document (si Index) o implementeaza
 * pentru a gestiona lista de observatori.
 */
```

- IObservable -- Stabilește ce operații trebuie să expună orice clasă observabilă

```
/**
 * Interfata Observable - Document (si Index) o implementeaza
 * pentru a gestiona lista de observatori.
 */
class IObservable {
public:
    virtual ~IObservable() = default;
    virtual void adaugaObserver(std::shared_ptr<IObserver> obs) = 0;
    virtual void eliminaObserver(std::shared_ptr<IObserver> obs) = 0;
protected:
    virtual void notificaObserveri(const EventCautare& ev) = 0;
};
```

- Logger — observer concret, moștenește IObserver și implementează logica concretă de înregistrare:

```
class Logger : public IObserver {
public:
    std::string caleFisierLog;
    bool afisareConsola;

    explicit Logger(const std::string& caleFisier = "search.log",
                   bool consola = false)
        : caleFisierLog(caleFisier), afisareConsola(consola) {}

    void onCautare(const EventCautare& ev) override {
        std::ostringstream linie;
        linie << "[" << ev.timestamp << "]" "
            << "TIP=" << ev.tipOperatie << " "
            << "QUERY=" << ev.interogare << " "
            << "REZULTATE=" << ev.rezultate.size();
        if (!ev.rezultate.empty()) {
            linie << " (";
            for (size_t i = 0; i < ev.rezultate.size(); ++i) {
                if (i) linie << ", ";
                linie << ev.rezultate[i];
            }
            linie << ")";
        }
        linie << "\n";

        // Scriere in fisier (append)
        std::ofstream log(caleFisierLog, std::ios::app);
        if (log.is_open()) log << linie.str();

        if (afisareConsola)
            std::cout << "[LOG] " << linie.str();
    }

    // Returneaza toate inregistrarile din fisierul de log
    std::vector<std::string> citesteLogs() const {
        std::vector<std::string> logs;
        std::ifstream f(caleFisierLog);
        std::string l;
        while (std::getline(f, l))
            if (!l.empty()) logs.push_back(l);
        return logs;
    }
};
```

- Document — entitate de bază + observable, reprezintă un fișier text:

```
class Document : public IObservable {
public:
    std::string caleFisier;    // "/home/user/documente/linux.txt"
    std::string continut;     // textul propriu-zis

    bool incarca();           // citeste fisierul de pe disc
    std::string numeFisier(); // returneaza doar "linux.txt"
    void emitEvent(const EventCautare& ev);

private:
    std::vector<std::shared_ptr<IObserver>> observeri;
};
```


- Index — clasa centrală + observable, coordonează toate operațiile: încărcare, indexare, căutare, notificare.

```
class Index : public IObservable {
public:
    // Structura principala: cuvant -> set de fisiere
    std::map<std::string, std::set<std::string>> indexInvers;

    std::vector<Document>    documente;
    StopWords                stopWords;

    bool incarcaDocumente(const std::string& director);
    void construiescIndex();

    std::set<std::string> caută(const std::string&, OperatieQuery);
    std::set<std::string> cautăSimpu(const std::string&);
    std::set<std::string> cautăAND(const std::vector<std::string>&);
    std::set<std::string> cautăOR(const std::vector<std::string>&);

private:
    std::vector<std::shared_ptr<IObserver>> observeri;
};
```

Concepte POO utilizate

- **Moștenire**

Logger moștenește IObserver, Document și Index moștenesc IObservable:

```
class Logger : public IObserver {
...
class Document : public IObservable {
...
class Index : public IObservable {
...
}
```

- **Polimorfism**

Observerii sunt stocați ca shared_ptr<IObserver>, deci notificarea funcționează indiferent de tipul concret:

```
void notificaObserveri(const EventCautare& ev) override {
    for (auto& o : observeri) o->onCautare(ev);
}
```

- **Design Pattern – Observer**

Implementarea completă a pattern-ului:

- Subject (Index) — menține lista de observeri și emite evenimente după fiecare căutare
- Observer (Logger) — reacționează la eveniment scriind în fișier
- Eveniment (EventCautare) — obiect de date care transportă contextul (interogare, tip, rezultate, timestamp)

- **Abstractizare**

Programul definește două interfețe pure care stabilesc contracte clare:

```
/**
 * Interfata Observer - orice clasa care vrea sa fie notificata
 * la o cautare trebuie sa implementeze onCautare().
 */
class IObservable {
public:
    virtual ~IObservable() = default;
    virtual void onCautare(const EventCautare& ev) = 0;
};

/**
 * Interfata Observable - Document (si Index) o implementeaza
 * pentru a gestiona lista de observatori.
 */
class IObservable {
public:
    virtual ~IObservable() = default;
    virtual void adaugaObserver(std::shared_ptr<IObservable> obs) = 0;
    virtual void eliminaObserver(std::shared_ptr<IObservable> obs) = 0;
protected:
    virtual void notificaObservatori(const EventCautare& ev) = 0;
};

/**
```

Funcțiile pot fi utilizate fără a se ști cum funcționează în spate.

4. Diagrama UML

